Tutorial 2 — Relational Data Models

BCL1223 / BIT1223 Database Systems

Question 1

a) Define the relational database model. (2 marks)

The relational database model, introduced by Codd (1970), organizes data into *relations* — two-dimensional tables composed of *tuples* (rows) and *attributes* (columns). Each relation represents an entity or relationship, with constraints such as primary keys ensuring tuple uniqueness and foreign keys enforcing referential integrity across relations. The model provides a logical abstraction that decouples data storage from data access through set-oriented operations defined in relational algebra.

b) Explain TWO advantages of the relational database model compared to hierarchical and network database models. (4 marks)

The first advantage is **logical simplicity and data independence**. Unlike hierarchical and network models that require programmers to navigate predefined pointer paths or parent–child structures, relational databases present all data as simple tables. Changes to physical storage structures — adding indexes or reorganizing files — do not alter the logical schema, shielding applications from storage-level modifications (Elmasri & Navathe, 2016).

The second advantage is **flexible ad-hoc querying** via declarative languages such as SQL. In hierarchical and network models, retrieving data requires writing procedural code that explicitly traverses record linkages. The relational model supports set-based operations (SELECT, PROJECT, JOIN) that allow users to express complex queries without specifying access paths, improving productivity and reducing development time.

c) Differentiate between logical data independence and physical data independence. (4 marks)

Aspect	Logical Data Independence	Physical Data Independence

Scope	Protects external schemas from changes to the conceptual schema	Protects the conceptual schema from changes to the internal storage
Examples of change	Adding a new column, splitting a table, modifying relationships	Changing file organization, adding indexes, altering storage hardware
Impact level	Higher-level abstraction — more difficult to achieve	Lower-level abstraction — relatively easier to implement
Typical difficulty	Requires view mapping, may involve restructuring applications	Handled transparently by the DBMS

Question 2

Given: STUDENT(StudentID, StudentName, Programme, CGPA)

a) Explain the meaning of the terms relation, tuple, and attribute using the STUDENT table above. (6 marks)

A **relation** corresponds to the entire STUDENT table — a named set of tuples sharing the same attribute structure. It is the fundamental building block of the relational model.

A **tuple** is an individual row within the relation. For example, the row `(S12345, Ali Bin Ahmad, Computer Science, 3.72)` constitutes one tuple representing a single student entity.

An **attribute** is a named column of the relation, such as `StudentID`, `StudentName`, `Programme`, or `CGPA`. Each attribute draws values from a defined domain (e.g., `CGPA` draws from a decimal range 0.00–4.00). The number of attributes in a relation is called its *degree* — four in this case.

b) Why is a table considered a logical representation of a relation? (4 marks)

A table mirrors the mathematical definition of a relation — a subset of the Cartesian product of attribute domains. Each row is an ordered n-tuple of domain values, and the column headers name each dimension of the

product. The table's properties (unique rows, unordered tuples, atomic attribute values) directly implement the characteristics of a mathematical relation, making it a natural logical representation.

Question 3

Given: EMPLOYEE(EmpID, EmpName, Department, Salary) and EmpID → EmpName, Department, Salary

a) Define functional dependency. (2 marks)

A functional dependency $X \rightarrow Y$ states that for any two tuples in the relation, if they share the same value for attribute set X, they must also share the same value for attribute set Y. The value of X uniquely determines the value of Y.

b) Explain why EmpID functionally determines the other attributes. (4 marks)

`EmpID` serves as the unique identifier for each employee. No two employees share the same `EmpID` — it is assigned uniquely at record creation. Given a specific `EmpID`, the system retrieves exactly one `EmpName`, one `Department`, and one `Salary`. This one-to-one correspondence satisfies the definition of functional dependency.

c) Determine whether EmpName can functionally determine EmpID. Justify your answer. (4 marks)

`EmpName` cannot functionally determine `EmpID` because employee names are not guaranteed unique. Two different employees may share the same name — for instance, two individuals named "Ahmad Faiz" would have different `EmpID` values. Under `EmpName → EmpID`, every occurrence of "Ahmad Faiz" would need to map to a single `EmpID`, which is impossible when the name is shared. Therefore, the dependency does not hold.

Question 4

Given: COURSE_REGISTRATION(StudentID, CourseID, Semester, Grade)

a) Define the following terms: (8 marks)

Term	Definition	Example
Superkey	A set of attributes that uniquely identifies every tuple in a relation	`{StudentID, CourseID, Semester}`; `{StudentID, CourseID, Semester, Grade}`
Candidate Key	A minimal superkey — no proper subset is itself a superkey	`{StudentID, CourseID, Semester}` (removing any attribute loses uniqueness)
Primary Key	The candidate key chosen as the main row identifier	`{StudentID, CourseID, Semester}`
Composite Key	A primary key consisting of two or more attributes	`(StudentID, CourseID, Semester)`

b) Identify the most appropriate primary key for the relation and justify your answer. (4 marks)

The most appropriate primary key is the composite key `(StudentID, CourseID, Semester)`.

Justification: A student can register for multiple courses, so `StudentID` alone is insufficient. A single course has many students, so `CourseID` alone is insufficient. Even the pair `(StudentID, CourseID)` is not enough because the same student may retake the same course in a different semester. Adding `Semester` resolves this ambiguity. `Grade` cannot be part of the key because it is the value being recorded, not an identifier.

Question 5

Given: DEPARTMENT(DeptID, DeptName) and EMPLOYEE(EmpID, EmpName, DeptID)

a) Identify the primary key and foreign key in the tables above. (4 marks)

Table	Primary Key	Foreign Key
DEPARTMENT	`DeptID`	—
EMPLOYEE	`EmpID`	`DeptID` (references DEPARTMENT.DeptID)

b) Explain the concept of referential integrity. (3 marks)

Referential integrity ensures that every foreign key value in a referencing table must either (i) match a primary key value in the referenced table or (ii) be NULL. This constraint guarantees that relationships between tables remain consistent — an employee cannot be assigned to a department that does not exist.

c) Describe what would happen if an employee record contains a DeptID that does not exist in the DEPARTMENT table. (3 marks)

Such a record would violate referential integrity and create an *orphan row* — an employee linked to a non-existent department. The DBMS enforces referential integrity at the constraint level; any INSERT or UPDATE that produces an invalid `DeptID` is rejected with an integrity constraint violation error.

Question 6

a) Convert the information into relational schemas. (4 marks)

```
CUSTOMER(CustomerID, CustomerName)
ORDER(OrderID, OrderDate, CustomerID*)
```

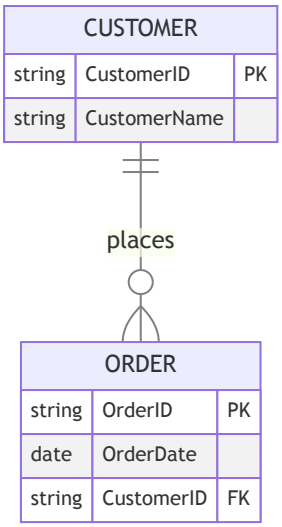
b) Underline the primary keys and indicate foreign keys. (4 marks)

```
CUSTOMER( CustomerID , CustomerName )
ORDER( OrderID , OrderDate, CustomerID* )
```

`CustomerID` is the primary key of CUSTOMER and a foreign key in ORDER.

c) Explain the relationship between the entities identified. (2 marks)

The relationship is **One-to-Many (1:M)** — one customer can place many orders, but each order belongs to exactly one customer. This is implemented by embedding the `CustomerID` foreign key inside the ORDER table.



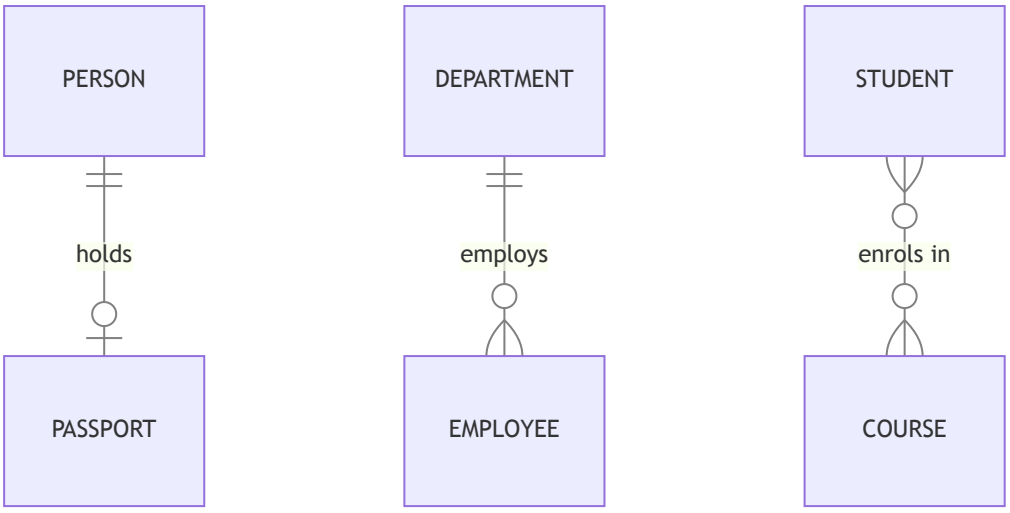
Question 7

a) Differentiate between the following relationship types: (6 marks)

Type	Definition
One-to-One (1:1)	One entity in A relates to at most one entity in B, and vice versa
One-to-Many (1:M)	One entity in A relates to many entities in B; each B relates to one A
Many-to-Many (M:N)	One entity in A relates to many in B, and one entity in B relates to many in A

b) Provide ONE real-world example for each relationship type. (6 marks)

- **1:1 — Person and Passport:** Each person holds at most one passport, and each passport belongs to exactly one person.
- **1:M — Department and Employee:** One department employs many employees; each employee works in exactly one department.
- **M:N — Student and Course:** A student enrols in multiple courses; each course contains many enrolled students. A junction table is required to resolve this.



c) Explain why M:N relationships cannot be directly implemented in a relational database. (3 marks)

Relational tables store data as rows with fixed attribute sets. An M:N relationship implies that one row in table A can be associated with an arbitrary number of rows in table B — storing this directly would require a variable-length attribute, violating the atomicity rule of first normal form (1NF). The standard solution is a *junction table* that decomposes the M:N into two 1:M relationships. For example, an `Enrolment` table with composite primary key `(StudentID, CourseID)` resolves the M:N between Student and Course.

Question 8

a) Define a data dictionary. (3 marks)

A data dictionary — also called a *system catalog* — is a centralized repository of metadata describing the structure, constraints, and properties of all database objects. It stores information such as table names, column definitions, data types, primary and foreign keys, indexes, views, user permissions, and storage parameters.

b) Explain the role of metadata in a relational database. (3 marks)

Metadata provides the DBMS with a self-describing framework for managing data. When a query is submitted, the DBMS consults the

metadata to resolve table and column names, verify access rights, check constraint compliance, and retrieve statistics for query optimization.

c) Discuss THREE benefits of maintaining a system catalog. (6 marks)

- **Data Consistency and Standardization:** The catalog enforces uniform naming conventions and data-type rules across all tables. Changes to a column definition are immediately visible to all users and applications, preventing schema drift.
- **Access Control and Security:** The catalog stores user privileges at the table, column, and operation levels. The DBMS checks these permissions on every query, ensuring only authorized users can access sensitive data.
- **Query Optimization:** The catalog maintains statistics — row counts, index selectivity, data distribution histograms — that the query optimizer uses to choose efficient execution plans (e.g., index scans versus full table scans).

Question 9

Given: STUDENT(StudentID, StudentName, Programme)

a) Explain the difference between the SELECT and PROJECT operations. (4 marks)

The **SELECT** operation (σ) filters *rows* (tuples) based on a predicate condition. It reduces cardinality but retains all attributes.

The **PROJECT** operation (π) filters *columns* (attributes), retaining only those specified and eliminating duplicate tuples. It reduces the degree but may not reduce cardinality unless duplicates arise.

b) Write the relational algebra expression to: (4 marks)

Retrieve all students from the "Computer Science" programme:

$$\sigma_{\texttt{Programme = "Computer Science"}}(\texttt{STUDENT})$$

Display only StudentID and StudentName:

$$\pi_{\texttt{StudentID, StudentName}}\left(\ \sigma_{\texttt{Programme = "Computer Science"}}(\texttt{STUDENT})\ \right)$$

c) Differentiate between UNION and INTERSECT with suitable examples. (4 marks)

Operation	Symbol	Result	Example
UNION	$R \cup S$	All tuples in R or S (or both), duplicates removed	$\pi_{\text{Programme}}(\text{STUDENT}) \cup \pi_{\text{Programme}}(\text{STAFF})$ returns all unique programmes across both tables
INTERSECT	$R \cap S$	Only tuples appearing in both R and S	$\pi_{\text{Programme}}(\text{STUDENT}) \cap \pi_{\text{Programme}}(\text{STAFF})$ returns programmes that exist in both tables

Both operations require operands to be *union-compatible* — same number of attributes with compatible domains.

Question 10

a) Define data redundancy. (2 marks)

Data redundancy refers to the unnecessary duplication of data values across multiple records or tables within a database. While controlled redundancy (e.g., foreign keys) is essential for representing relationships, uncontrolled redundancy wastes storage and creates maintenance problems.

b) Explain TWO data anomalies that may result from excessive redundancy. (4 marks)

- **Update Anomaly:** A change to a redundant value must be applied to every occurrence. If a student's address is stored in both ENROLMENT and STUDENT, changing it requires updating many rows across both tables. A missed update leaves the database inconsistent.
- **Deletion Anomaly:** Deleting a row may unintentionally remove the only copy of some other fact. If course details are stored redundantly within an enrolment record, deleting the last enrolment for a course also deletes the course's name and description.

c) Describe the purpose of indexing in a relational database. (2 marks)

An index is an auxiliary data structure (commonly a B^+-tree) that provides fast access paths to rows based on column values. It sacrifices write performance and storage space to accelerate read operations.

d) Explain how indexes improve query performance. (4 marks)

Without an index, the DBMS must perform a *full table scan* — reading every row from disk to locate those matching a WHERE clause. An index allows the DBMS to navigate directly to relevant disk pages using a balanced-tree search, reducing the search from $O(n)$ to $O(\log n)$ comparisons. Indexes also speed up JOIN operations by enabling fast foreign-key lookups and support efficient ORDER BY and GROUP BY clauses by scanning the index in sorted order.

References

1. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. https://doi.org/10.1145/362384.362685
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.